

امنیت و ارزیابی عامل‌های کدنویسی شهودی

اعتماد مؤثر در دو محور: ماندن درون مرز، و ارزش داشتن آنچه درون مرز رخ داد

مطالعه پیشنهادی: ۵۰ دقیقه سطح: پیشرفته

فهرست مطالب

- مقدمه: دو محور اعتماد
- هفت ستون معماری امنیت عامل
- جعبه‌شن و دفاع زنجیره تأمین
- امن‌سازی منطق برنامه
- هویت، اعتماد و کنش‌های پرمخاطره
- سه‌گانه قرمز، آبی و سبز
- رصدپذیری: ممیزی ذهن عامل
- جمع‌بندی امنیت
- ارزیابی: چرا متفاوت است
- چه چیزی را بسنجیم: هفت بُعد
- چگونه بسنجیم: هفت روش
- نکته‌های کاربردی
- خودآزمایی

۱. مقدمه: دو محور اعتماد

تبدیل مدل‌های هوش مصنوعی به عامل‌های سازمانی امن و هم‌راستا، از راه اعتماد پیوسته و ارزیابی دقیق، بیش از هر زمان دیگری اهمیت دارد.

توسعهٔ پرسرعت و قصدمحور نوآوری را به شدت شتاب می‌دهد، اما پارادایم‌های سنتی اعتماد را می‌شکند. در نرم‌افزار قطعی، اعتماد دوتایی است: کد کامپایل می‌شود، آزمون‌ها می‌گذرند، اعتبارنامه‌های ایستا معتبرند. در سامانهٔ عامل‌محور، نیروی کاری خودگردان **اختیار محیطی** دارد: کد تولیدشده را اجرا می‌کند، به رابط‌های داخلی حساس دسترسی می‌گیرد و محیط‌های تولید را پویا تغییر می‌دهد.

بازتعریف اعتماد در دو محور

امنیت به شما می‌گوید آیا عامل درون مرز ماند و ایمن و بدون قصد مخرب عمل کرد.
ارزیابی به شما می‌گوید آیا آنچه درون آن مرز رخ داد ارزش انتشار دارد.

عامل کدنویسی‌شده می‌تواند از هر واریسی امنیتی رد شود و همچنان قصد توسعه‌دهنده را بنیاداً بد بفهمد، قراردادهای پروژه را نادیده بگیرد، یا تجربهٔ کاربر را بی‌سروصدا خراب کند.

اعتماد مؤثر

آزمون و مدل‌های امنیتی سنتی بر منطق قطعی تکیه دارند: مجموعه‌ای ثابت از ورودی‌ها، خروجی قابل‌پیش‌بینی تولید می‌کند. اما در سامانهٔ عامل‌محور، عامل می‌تواند توکن دسترسی معتبری داشته باشد و در عین حال با **قصد ناهم‌راستا** خودگردان عمل کند. یادآوری حیاتی: **مدل خام هوش مصنوعی، عامل نیست.** فقط وقتی عامل می‌شود که در «مهار» پیچیده شود که به آن وضعیت، اجرای ابزار، حلقهٔ بازخورد و قیدهای قابل‌اعمال می‌دهد. امن‌سازی این پارادایم یعنی جابه‌جا کردن تمرکز از امن‌سازی نحو کد به امن‌سازی همین مهار.

در این محیط سیال و غیرقطعی، هویت ایستا محیط دفاعی ضعیفی است. اعتماد دیگر نمی‌تواند دروازه‌ای باشد که عامل یک‌بار هنگام استقرار از آن رد می‌شود؛ باید پیوسته کسب، راستی‌آزمایی و بر پایهٔ زمینهٔ زمان اجرا اعمال شود. این اطمینان مستمر را **اعتماد مؤثر** می‌نامیم: سنج‌های پیوسته که در زنجیرهٔ تأمین، هویت، رفتار زمان اجرا و پیوندهای زمینه‌ای عامل ارزیابی می‌شود.

۲. هفت ستون معماری امنیت عامل

در محیط‌های سنتی سازمانی امنیت قطعی است: برنامه‌ها بر نحو قابل‌پیش‌بینی تکیه دارند و دسترسی با مدل‌های ایستای «هویت به‌مثابه محیط» اداره می‌شود. محیط عامل‌محور این مدل را بنیاداً به هم می‌ریزد. چون شکست‌های روزمرهٔ عامل اغلب به ابزاری غایب، قاعده‌ای مبهم یا حفاظی نبوده برمی‌گردند، سازمان‌ها باید به مدل «**زمینه به‌مثابه محیط**» کوچ کنند. و چون باید فرض کنیم مدل زیرین ممکن است شکست بخورد یا به خطر بیفتد، امنیت نمی‌تواند صرفاً درون خودِ هوش مصنوعی ساکن باشد؛ باید «پاکت ایمنی» بیرونی و سخت‌گیرانه‌ای بر آن تحمیل کنیم.

ستون	مالک سازمانی	تهدید اصلی و کنترل کلیدی
۱. زیرساخت و شبکه	مهندسان زیرساخت ابری	آلودگی بالادست و فرار از ظرف. اجرای زمان اجرا در جعبه‌شن‌های زودگذر سطح هسته ایزوله می‌شود و حکمرانی سخت‌گیرانه خروج شبکه تضمین می‌کند داده تولیدشده فقط از راه حافظه‌های نهان مجاز یا پروکسی‌های داخلی صریح سفر کند.
۲. داده	معماران داده	نشت اطلاعات حساس از پنجره زمینه و بلعیدن داده آلوده بازیابی‌شده. داده در سکون با کلیدهای مدیریت‌شده مشتری و داده در انتقال با mTLS محافظت می‌شود. انبارهای حافظه بلندمدت – به‌ویژه پایگاه‌های بُرداری – باید افزاز سخت‌گیرانه مستأجران را اعمال کنند تا آلودگی بُرداری میان‌مستأجری رخ ندهد.
۳. مدل	مهندسان هوش مصنوعی	حملات معنایی که دستورهای مدل را واژگون می‌کنند. در گردش‌کار عامل‌محور، درخواست و فایل‌های قاعده کد منبع تازه‌اند و باید مثل مصنوع‌های بسیار حساس و گواهی‌شده رمزنگاشتی با آن‌ها رفتار شود.
۴. برنامه و زمان اجرا	توسعه‌دهندگان عامل	خودگردانی مهارنشده هنگام اجرای منطق و استفاده از ابزار. دیواره آتش مبتنی بر قاعده سنتی کافی نیست: دیواره آتش مدل‌زبانی برای پالایش پویا، قلاب‌های قطعی در نقاط چرخه عمر، و دروازه متمرکز عامل که حرکت جانبی غیرمجاز را مهار می‌کند.
۵. هویت و دسترسی	مدیران هویت	مسئله معاون گیج. حل با هویت یکتای رمزنگاشتی برای هر عامل، کنترل دسترسی مبتنی بر ویژگی، و کاهش دامنه به‌هنگام توکن – یعنی ماتریس مجوزی از جنس قصد × کاربر × زمان.
۶. رصدپذیری و عملیات امنیت	تیم‌های عملیات امنیت	«شکست‌های نامرئی» که در آن عامل بی‌سروردا وارد حلقه استدلال بی‌پایان می‌شود. راه‌حل: استقرار سه‌گانه خودگردان تیم قرمز، آبی و سبز.

ستون	مالک سازمانی	تهدید اصلی و کنترل کلیدی
۷. حاکمیت	افسران حاکمیت و انطباق	مسئولیت حقوقی تصمیم‌گیری خودکار. نیازمند نظارت پیوسته، اولویت‌بندی مخاطره بر پایه اثر کسب‌وکاری، و رد ممیزی تغییرناپذیر که هر کنش واقعی را به عاملی مشخص و انسانی که مستقر یا تأییدش کرده نسبت می‌دهد. همچنین جایگزینی دکمه‌های ساده تأیید با «بازبینی منطق» اجباری.

۳. جعبه‌شن و دفاع زنجیره تأمین (ستون‌های ۱ و ۴)

حلقه شهودی و جعبه‌شن زودگذر

سازوکار هسته کدنویسی شهودی، ترجمه پویا و درجای قصد انسانی به منطق اجراپذیر است. اما عامل‌ها به‌ندرت در نخستین تلاش کد بی‌نقص می‌نویسند. واقعیت توسعه قصدمحور، چرخه‌ای پرسرعت و تکرارشونده است: عامل اسکرپتی می‌نویسد، اجرا می‌کند، گزارش‌های خطا را می‌خواند و منطق را خودگردان بازنویسی می‌کند. چون این فرایند مولد تغییرپذیری بالایی وارد می‌کند، کد حاصل را نمی‌توان به‌طور ضمنی معتبر دانست.

جعبه‌شن فقط زندان نیست

هر کدی که مهارت تولید می‌کند باید نخست در جعبه‌شنی زودگذر و ایزوله از شبکه اجرا شود. این جعبه‌شن‌ها باید فعالانه دسترسی خام به میزبان را مسدود کنند و وضعیتشان را میان اجراها کاملاً بازنشانی کنند؛ تا حتی اگر اسکرپیت شهودی آسیب‌پذیری شدیدی داشته باشد یا برای فرار از ظرف دستکاری شده باشد، منطق به‌خطراتده نتواند ماندگار شود یا به گره میزبان آسیب بزند.

مهار بسته‌های توهمی

تولید کد عامل‌محور یک آسیب‌پذیری بسیار مشخص و خطرناک زنجیره تأمین وارد می‌کند: مدل‌های زبانی مرتب بسته‌های نرم‌افزاری‌ای را نام می‌برند که وجود ندارند. مهاجمان فعالانه انجمن‌های توسعه‌دهندگان و خروجی‌های هوش مصنوعی را برای یافتن این توهم‌ها رصد می‌کنند و پیشاپیش بدافزار را با دقتاً همان نام‌های ساختگی منتشر می‌کنند. چون عامل‌های خودگردان می‌توانند گراف وابستگی را بدون تأیید انسان تغییر دهند، یک توهم کافی است تا بدافزار مستقیم وارد محیط ساخت شود. این تکنیک با نام سوءاستفاده از نام توهمی شناخته می‌شود.

دفاع سه‌لایه:

- تأمین وابستگی‌ها منحصرأً از تأمین‌کنندگان بررسی‌شده یا فهرست‌های داخلی سازمان.
- اعمال قفل نسخه رمزنگاشتی سخت‌گیرانه.
- راستی‌آزمایی خودکار صورت‌ریز اجزای نرم‌افزار و امضاهای دیجیتال در خط لوله، پیش از هر ارتقا به تولید – به‌عنوان دروازه‌ای قطعی.

حکمرانی خروج شبکه و دسترسی غیرتعاملی

در نرم‌افزار سنتی، ترافیک خروجی بسیار قابل‌پیش‌بینی است. در سامانه‌های شهودی، خروج غیرقطعی است چون با استفاده پویا از ابزارهای تازه‌تولیدشده هدایت می‌شود. حالت شکست رایج این است که عامل ناخواسته بکوشد کد راستی‌آزمایی‌نشده را به محیط زنده بفرستد یا داده حساس را خارج کند.

چرا فهرست مجاز کافی نیست

فهرست دامنه‌های تأییدشده نمی‌تواند عامل را در برابر تزریق‌های غیرمستقیم درخواست که در صفحه‌های وب شخص ثالث پنهان‌اند محافظت کند. راه‌حل: محدودکردن عامل به دسترسی غیرتعاملی به اینترنت. مدیران باید عامل را وادار کنند اطلاعات بیرونی را منحصرأً از راه حافظه‌های نهان برون‌خط یا سرویس‌های خزش وب اختصاصی و پیش‌پالایش‌شده بگیرد.

۴. امن‌سازی منطق برنامه (ستون ۴)

چون کدنویسی شهودی ذاتاً کارکرد فوری را بر طراحی امن ترجیح می‌دهد، برنامه‌های تولیدشده مرتب نقص‌های ساختاری شدیدی دارند. کاربران اغلب کد تولیدشده را صرفأً به این دلیل که کامپایل می‌شود و بدون خطا اجرا می‌شود ضمناً معتبر می‌دانند، و همین آن‌ها را نسبت به این واقعیت کور می‌کند که برنامه ممکن است کنترل‌های استاندارد امنیتی پشتیبان را کاملاً دور زده باشد.

دو الگوی شکست

اول، اعتماد بیش‌ازحد به مرورگر. تولید خودکار معمولاً کم‌مقاومت‌ترین مسیر را می‌رود و عملیات حساس را در سمت کاربر انجام می‌دهد. به‌جای هدایت از راه سروری امن، کد تولیدشده اغلب کلیدهای API، اعتبارسنجی گذرواژه و پرچم‌های نشست کاربر را مستقیم در سمت مرورگر می‌ریزد. یعنی هرکس ابزار توسعه مرورگرش را باز کند می‌تواند این اعتبارنامه‌ها را بخواند یا سطح دسترسی‌اش را دست‌کاری کند، بی‌آنکه هرگز به گذرواژه واقعی نیاز داشته باشد.

دوم، پشتیبان رهاشده. سرعت ساخت این برنامه‌ها از سرعت برپاکردن لایه‌های نامرئی امنیت جلو می‌زند. ابزارها در وصل‌کردن پایگاه داده یا بالاآوردن داشبورد مدیریت عالی‌اند، اما به‌ندرت کنترل‌های دسترسی سخت‌گیرانه و «رد

پیش‌فرض» لازم برای محافظت از آن‌ها را فعال می‌کنند. در نتیجه چیزهای پایه‌ای مثل امنیت سطح ردیف پایگاه داده از قلم می‌افتند و داده‌ی خصوصی کاربران و محیط‌های داخلی کاملاً روی اینترنت باز می‌مانند.

آشتی اصطکاک ویرایشگر با اعمال در خط لوله

برای گرفتن این نقص‌های ساختاری باید میان سرعت توسعه‌دهنده و اعمال سخت‌گیرانه امنیت توازن برقرار کرد. تلاش برای مسدودسازی تهاجمی درخواست‌های ناامن مستقیماً در ویرایشگر، به آسانی دور زده می‌شود و برای توسعه‌دهنده بی‌خطری که روی منطق پیچیده تکرار می‌کند اصطکاک بیش‌ازحد می‌سازد. به جایش، «انتقال به چپ» باید با **وارسی‌گرهای مشورتی** در ویرایشگر پیاده شود که راهنمایی زمان‌واقعی می‌دهند، در حالی که اعمال بی‌اغماض امنیت به واری‌های قطعی درون خط لوله منتقل می‌شود: آزمون امنیت ایستای برنامه و تحلیل ترکیب نرم‌افزار.

جعل سرور ابزار و مجوزدهی زمینه‌ای

عامل‌ها بیش‌ازپیش به چارچوب‌های هماهنگی ابزار تکیه می‌کنند که اجازه می‌دهند در زمان اجرا سرورهای بیرونی یا داخلی را کشف کنند و به آن‌ها وصل شوند. این یک بردار تهدید حیاتی وارد می‌کند: سروری جعلی یا به‌خطرافتاده می‌تواند خود را ابزاری مشروع جا بزند تا بار مخرب تزریق کند یا امتیاز بیش‌ازحد بخواهد. و چون عامل‌ها خودگردان کار می‌کنند، ممکن است فرمان‌های مخرب این سرورها را پیش از هر مداخله انسانی اجرا کنند.

راه‌حل دولایه است: **دیواره آتش مدل‌زبانی** در زمان اجرا و پیش از عامل فعال، برای رهگیری پویای تزریق‌های فرصت‌طلبانه؛ و **دروازه متمرکز عامل** که مجوزدهی زمینه‌ای را ارزیابی می‌کند و پویا راستی‌آزمایی می‌کند که آیا درخواست عامل برای فراخوانی یک ابزار، دقیقاً با قصد اولیه توسعه‌دهنده هم‌راستاست یا نه.

۵. هویت، اعتماد و کنش‌های پرمخاطره (ستون ۵)

چون توسعه‌دهندگان اغلب از زبان طبیعی مبهم یا بسیار انتزاعی برای تولید کد استفاده می‌کنند (مثلاً «مسیریابی پشتیبان را درست کن»)، گردش کارهای عاملی حاصل ذاتاً گسترده‌اند. دادن هویت‌های سرویس مشترک و بلندعمر به این عامل‌های خودگردان، بردار تهدید داخلی مدیریت‌ناپذیری می‌سازد.

معاون گیج: هویت تفویضی در برابر هویت عامل‌محور

حتی با هویت یکتا، عامل شهودی همچنان بسیار مستعد مسئله معاون گیج است: تزریق درخواست – مثلاً دستوری مخرب پنهان در مخزنی متن‌باز که توسعه‌دهنده ناآگاهانه در پنجره زمینه ویرایشگرش چسبانده – عاملی با امتیاز بیش‌ازحد را فریب می‌دهد تا فرمان غیرمجازی را به نیابت مهاجم اجرا کند.

عامل هرگز نباید داور نهایی دسترسی باشد. به جای کارکردن زیر اعتبارنامه‌های تفویض‌شده کاربر انسانی – که به عامل دسترسی محیطی خطرناک می‌دهد – عامل باید با هویتی اختصاصی که صریحاً عامل محور برچسب خورده احراز هویت کند. هویت متمایز و قابل مشاهده تضمین می‌کند مجوزهایش محدود و مشمول گزارش ممیزی ریزدانه بماند.

اختیار محیطی صفر و کاهش دامنه به‌هنگام

معماری باید اختیار محیطی صفر را اعمال کند: عاملی که یک «حس و حال» را اجرا می‌کند هرگز نباید امتیازهای کامل و محیطی مدیریتی توسعه‌دهنده را به ارث ببرد. وقتی عامل پویا اسکریپت یا مهارتی تازه می‌نویسد، جعبه‌شن اجرا اعتبارنامه‌ای تازه و به‌شدت محدود می‌گیرد که صریحاً به همان منابع داده لازم برای همان اسکریپت محدود است. افزون بر این، مدیران باید فهرست‌های مجاز درخت فایل را اعمال کنند تا عملیات خواندن و نوشتن به دایرکتوری‌های مشخص پروژه محدود بماند، با قواعد رد پیش‌فرض برای مسدودکردن دسترسی به رازها، اسکریپت‌های ساخت و مانیفست‌های تولید. این توکن‌ها به‌شدت زودگذرند و دقیقاً همان لحظه‌ای که کار تمام می‌شود منقضی می‌شوند.

درخواست تأیید، چالش سخت‌افزاری و تفاوت‌نمای شهودی

کنش‌های پرمخاطره – مانند تغییر پایگاه داده تولید، اجرای انتقال مالی، یا تغییر پیکربندی دسترسی – به راستی آزمایشی صریح نیاز دارند و نمی‌توانند به دکمه‌های ساده «تأیید/رد» تکیه کنند. چون کدنویسان شهودی اغلب برای نوشتن نحو پیچیده‌ای که کاملاً نمی‌فهمند به هوش مصنوعی تکیه می‌کنند (مغالطه «کار می‌کند، بفرستش»)، دروازه‌های ساده تأیید سریع به خستگی تأیید می‌انجامد و توسعه‌دهنده کورکورانه کدی را مجاز می‌کند که درکش نمی‌کند. پاسخ، درخواست تأیید ساختارمند و زمینه‌آگاه است، همراه با دو مرز امنیتی متمایز:

- **احراز هویت چندعاملی سخت‌افزاری و رمزنگاشتی:** سامانه باید چالش فیزیکی را الزامی کند؛ مثلاً لمس کلید امنیتی سخت‌افزاری برای تأیید رمزنگاشتی اجرا.
- **تفاوت‌نمای شهودی:** پیش از اجرای ابزار حیاتی، حدنصابی از ارزیاب‌ها درخواست را رهگیری می‌کنند و کد پیچیده تولیدشده را به خلاصه‌ای به زبان ساده برمی‌گردانند. این تفاوت‌نما به توسعه‌دهنده نشان می‌دهد قصد مبهم اولیه‌اش دقیقاً به چه گام‌های اجرایی نگاشت می‌شود، و تضمین می‌کند که اپراتور انسانی واقعاً می‌فهمد چه چیزی را تأیید می‌کند.

۶. سه‌گانه قرمز، آبی و سبز (ستون ۶)

در محیط شهودی، منطق برنامه با سرعتی بی‌سابقه تولید، اجرا و دور ریخته می‌شود. چون سطح حمله غیرقطعی و با زبان طبیعی هدایت‌شده است، عملیات امنیتی سنتی و دستی نمی‌تواند همپا بماند. برای امن‌کردن سامانه‌های

خودگردان، خود عملیات امنیتی باید عامل محور شود: سه‌گانه‌ای پیوسته و هوش مصنوعی محور که موازی با گردش کار توسعه‌دهنده اجرا می‌شود.

بارهای مخرب نامرئی و آلودگی مخزن

خود مخزن‌ها بردار حمله بسیار مؤثری‌اند. مهاجمان می‌توانند با تزریق نویسه‌های یونیکد با عرض صفر یا نویسه‌های هم‌ریخت مستقیم در کدپایه، مخزن را به خطر بیندازند. این بارهای مخرب نامرئی «در دید آشکار پنهان می‌شوند و از بازیابی انسانی می‌گریزند». و چون عامل‌ها کد را بسیار سریع‌تر از توسعه‌دهنده انسانی دست‌کاری و تکثیر می‌کنند، یک بار مخرب پنهان می‌تواند ظرف چند دقیقه در صدها فایل پخش شود.

تیم قرمز – عامل مهاجم

پایش منفعل بنیاداً واکنشی است. برای کشف آسیب‌پذیری‌های معنایی و بارهای نامرئی پیش از دشمنان بیرونی، سازمان‌ها باید عامل‌های تیم قرمز مجازی مستقر کنند. به جای اجرای آزمون‌های نفوذ ایستا، عامل مهاجم فعالانه «حس و حال خصمانه» تزریق می‌کند: ساخت پویای فرارهای پیچیده به شکل ایفای نقش، و دفن دستورهای مخرب پنهان درون بلوک‌های عظیم زمینه‌بازی‌شده یا پست‌های ساختگی انجمن – همان‌هایی که توسعه‌دهندگان مرتب در ویرایشگرشان می‌چسبانند. تیم قرمز فعالانه می‌آزماید که آیا عامل هدف با زمینه آلوده منحرف می‌شود و راه‌حلی ناامن توهم می‌کند یا نه.

تیم آبی – عامل مدافع

تیم آبی خودکار، جایگزین تحلیل رفتاری سنتی کاربر و موجودیت می‌شود که برای هوش مصنوعی غیرقطعی بی‌اثر است. عامل مدافع بر تحلیل رفتاری عامل تکیه می‌کند تا مسیرهای اجرای مورد انتظار را مبنا بگیرد و ناهنجاری‌های ویژه هوش مصنوعی را تشخیص دهد. این تیم پیوسته صورت‌ریز اجزای زمان اجرای عامل را پایش می‌کند: سیاهه‌ای پویا از ابزارها، مدل‌ها و منابع داده فعال در هر میلی‌ثانیه. اگر منطق عامل منحرف شود – مثلاً اسکریپتی ناگهان شروع به پرس‌وجوی تعداد غیرعادی ابزار بیرونی کند یا وارد حلقه منابع بی‌کران شود – موتور تحلیل فوری انحراف را علامت می‌زند.

تیم سبز – عامل ترمیم‌گر

وقتی تیم آبی عاملی به خطراتاده را تشخیص می‌دهد، روش‌های سنتی پاسخ به حادثه – مانند کشتن خشن ظرف میزبان – بسیار مخل و خطرناک‌اند: پایان دادن به عامل در میانه اندیشه می‌تواند رابط‌های متصل را در وضعیت خراب رها کند. به جای تیم سبز خودکار، قرنطینه باحالت اجرا می‌کند: دسترسی ابزار عامل با ظرافت سلب می‌شود و توانایی کنشش روی جهان منجمد می‌گردد، در حالی که حافظه کوتاه‌مدتش کاملاً دست‌نخورده برای تحلیل قانونی باقی می‌ماند. تیم سبز یک گام فراتر هم می‌رود و بازسازی خودکار انجام می‌دهد: اسکریپت ناامن را خودگردان

بازنویسی می‌کند تا آسیب‌پذیری را وصله کند و کد امن جایگزین را مستقیم در ویرایشگر به توسعه‌دهنده نشان می‌دهد.

یکپارچه‌سازی سه‌گانه و اجبار دسته‌های کوچک

برای جلوگیری از تولید تغییرهای عظیم و بازبینی‌ناپذیر، توسعه‌دهندگان باید خروجی عامل را به دسته‌های کوچک محدود کنند. بهترین راه، حلقهٔ آزمون‌محور است که در آن سامانه مانع می‌شود عامل هم‌زمان آزمون و کد پیاده‌سازی را تغییر دهد تا آزمون، مبنای عینی بماند. با این قیده‌ها، سه‌گانه رفتار عامل اصلی را در زمان اجرا در سه فاز تنظیم می‌کند:

- **فاز برنامه‌ریز:** وقتی عامل اصلی گردش‌کاری طراحی می‌کند، مهارتی تخصصی برای مدل‌سازی تهدید کم‌کم می‌کند نقص‌های منطقی و نقض سیاست را پیش از شروع اجرای فعال شناسایی کند.
- **فاز ارزیاب:** حدنصاب ارزیاب‌ها رد اجرای پیشنهادی را بازبینی می‌کند، در حالی که عامل مدافع هم‌زمان صورت‌ریز اجزا را راستی‌آزمایی و زمینهٔ معنایی را برای انحراف قصد پایش می‌کند.
- **فاز مجری:** هنگام اجرای کنش کاهش‌دامنه‌یافته، عامل ترمیم‌گر اجرای واقعی ابزار را پایش می‌کند و آمادهٔ فعال‌کردن فوری قرنطینهٔ باحالت یا حلقهٔ بازسازی خودکار است.

۷. رصدپذیری: ممیزی ذهن عامل (ستون‌های ۶ و ۷)

قاعدهٔ بنیادی

چیزی را که نمی‌بینید نمی‌توانید امن کنید. در ریزسرویس‌های سنتی، پاسخ موفق نشانهٔ عملیات موفق است. اما در سامانهٔ عامل‌محور، وضعیت «موفق» ممکن است صرفاً پوششی باشد بر سناریویی که در آن منطق درونی عامل بی‌سروصدا وارد حلقهٔ توهم شده است. این مخاطرهٔ حیاتی **انکار کیف پول** را وارد می‌کند: مهاجم عمداً حلقه‌های بی‌پایان و پرهزینه را راه می‌اندازد تا صورت‌حساب ابری و مدل‌زبانی سازمان را منفجر کند.

ردیابی مسیر شهودی و پویش محتوا

برای پاسخ به پرسش حیاتی «چرا عامل این کار را کرد؟»، تیم امنیت باید عدسی یکپارچه و زمان‌ترتیبی برای دیدن گام‌های شناختی عامل بسازد. با استفاده از چارچوب‌های استاندارد تله‌متری، می‌توان سیگنال‌های گوناگون – فراخوان‌های رابط، ورودی و خروجی ابزارها، بازبینی‌ها و تأخیر توکن – را در یک **مسیر شهودی** کامل جمع کرد. ردیابی این مسیر یعنی ثبت جهش شناختی عظیم از درخواست اولیهٔ کاربر تا درخت نحو انتزاعی کامپایل‌شده. برای استحکام بیشتر، این ردیابی باید با **پویش متمرکز محتوا** جفت شود که صریحاً برای بازرسی همهٔ قطعه‌کدها و اسکریپت‌های پویایی که عامل در زمان اجرا می‌گیرد طراحی شده است.

سنجش انحراف قصد و زوال اعتماد

همان‌طور که عامل پویا منطق تولید می‌کند و ابزار تازه می‌گیرد، محیط امنیتی‌اش دائم نوسان می‌کند و سیاهه‌های ایستای دارایی فوری کهنه می‌شوند. به جای آن‌ها، سکوه‌های رصدپذیری باید **صورت‌ریز اجزای زمان اجرا** را پایش کنند: سندی زنده که شعاع اثر فعال عامل را در هر میلی‌ثانیه نگاشت می‌کند.

و چون اعتماد در سامانه‌ای خودگردان دارایی مستهلک‌شدنی است، معماری پیوسته **انحراف قصد** را پایش می‌کند. اصل **زوال اعتماد** می‌گوید اعتماد وقتی از دست می‌رود که زنجیره اندیشه درونی عامل زیرهدف‌هایی را دنبال کند که از حس‌وحال اولیه انسان واگرا می‌شوند. برای نمونه، درخواست ساده‌ای مانند «پرس‌وجوی پایگاه داده را بهینه کن» ممکن است به تلاش عامل برای دانلود کتابخانه‌ای نمایه‌ساز و غیرمجاز منحرف شود.

نقاط بازگشت و قطع‌کننده‌های باحالت

برای جلوگیری از کنش‌های مخرب هنگام این انحراف، ستون رصدپذیری باید وضعیت را فعالانه مدیریت کند: پیش از آنکه عامل هر تغییری در کدپایه اجرا کند، سامانه باید نقطه بازگشتی در کنترل نسخه بسازد. هنگام ارزیابی مسیر شهودی در برابر صورت‌ریز اجزا، هر ناپایداری تشخیص‌داده‌شده فوری **امتیاز اعتماد پویای عامل** را جریمه می‌کند. اگر این امتیاز زیر آستانه از پیش تعریف‌شده بیفتد، «قطع‌کننده» ای خودکار فعال می‌شود: محیط از نقطه بازگشت استفاده می‌کند تا تغییرها را فوری برگرداند، دسترسی ابزار را با ملایمت سلب کند و اجرای خودگردان را منجمد کند – بدون خراب‌کردن رابط‌های متصل و با حفظ وضعیت محیط برای تحلیل قانونی.

۸. جمع‌بندی امنیت

محیط را به چپ ببرید

منابع معتبر و فهرست‌های داخلی راستی‌آزمایی‌شده را الزامی کنید. مسدودسازی در ویرایشگر گام اول مشورتی است؛ اتکای اصلی بر واری‌های قطعی در چند نقطه از خط لوله باشد.

حلقه شهودی را جعبه‌شن کنید

اسکرپت‌های تولیدشده پویا همیشه در جعبه‌شن سطح هسته و ایزوله از شبکه اجرا شوند. تحلیل ترکیب نرم‌افزار به‌روز را برای پویش وابستگی‌های توهمی یا آسیب‌پذیر پیش از رسیدن کد به تولید تعبیه کنید.

اختیار محیطی صفر را اعمال کنید

هرگز «کلید سراسری» ندهید. هویت‌های اختصاصی و توکن‌های به‌هنگام به‌شدت محدود که با پایان کار منقضی می‌شوند. برای کنش‌های پرمخاطره، دکمه تأیید کور را با تفاوت‌نمای شهودی جایگزین کنید.

عملیات امنیتی عامل‌محور مستقر کنید

پیوسته با عامل‌های تیم قرمز مجازی معماری را زیر فشار بگذارید، با تحلیل رفتاری صورت‌ریز پویا را پایش کنید، و به تیم سبز اختیار بازسازی خودکار بدهید.

مسیر اجرا را ردیابی کنید

فراخوان‌ها، ورودی ابزارها و گام‌های استدلال را ثبت کنید. پیوسته گزارش‌ها را برای رفتار غیرمنتظره پایش کنید و از نقاط بازگشت کنترل نسخه برای سلب دسترسی هنگام انحراف استفاده کنید.

۹. ارزیابی: چرا متفاوت است

کنترل‌های امنیتی محدود می‌کنند عامل **چه کاری می‌تواند بکند**. اما به پرسشی که توسعه‌دهنده واقعاً دارد پاسخ نمی‌دهند: آیا عامل همانی را ساخت که خواستم، و اصلاً خوب است؟ ارزیابی عامل‌های کدنویسی شهودی نه مثل ارزیابی نرم‌افزار قطعی است و نه مثل ارزیابی عامل خدمات مشتری یا عامل پژوهشی. سه چیز آن را یکتا می‌کند:

- ۱. شکاف کم‌مشخصگی (مشخصاتی وجود ندارد).** آزمون نرم‌افزار سنتی بر این فرض بی‌اغماض کار می‌کند که مشخصاتی کامل و سخت پیش از ارزیابی هر خط کد وجود دارد. کدنویسی شهودی دقیقاً برعکس است: درخواست زبان طبیعی کاربر ذاتاً ناقص است. «داشبورد را سریع‌تر کن» یک پرونده آزمون نیست. درخواست کاملاً به دانش نهفته، قضاوت زیبایی‌شناختی و تخصص حوزه‌ای مدل پایه تکیه می‌کند تا شکاف‌های عملیاتی را پر کند. نخستین کار ارزیابی این است که بسنجد آیا عامل این شکاف را با موفقیت پر کرد و مشخصات نانوشته درست را بازسازی کرد.
- ۲. کاربر اغلب نمی‌تواند خروجی را اعتبارسنجی کند.** کاربران غیرفنی نمی‌توانند ۶۰۰ خط کد را خطبه‌خط بازبینی کنند و مهندسان باتجربه هم در زمان واقعی نمی‌توانند. فاصله میان «عامل فکر می‌کند موفق شد» و «کد واقعاً درست است» اینجا از هر دسته دیگری بیشتر است و بستن همین فاصله، کار مرکزی ارزیابی است.
- ۳. نشست تکرارشونده است و کدپایه، وضعیت.** هر نوبت فایل‌های واقعی را تغییر می‌دهد و تصمیم‌های بد اولیه انباشته می‌شوند. ارزیابی باید نه فقط تصمیم‌های تک‌نوبتی بلکه کل قوس یک گفت‌وگوی چندنوبتی را، روی کدپایه‌ای زنده با قراردادهای وابستگی‌ها و تاریخچه خودش، پوشش دهد.

۱۰. چه چیزی را بسنجیم: هفت بُعد

ابعاد ارزیابی در دو گروه جای می‌گیرند؛ به‌علاوه یک محور عرضی.

ابعاد کاربرمحور

#	بُعد	شرح
۱	رضایت قصد	آیا عامل همانی را ساخت که کاربر منظور داشت ، نه فقط آنچه گفت؟ سخت‌ترین بُعد برای ارزیابی، چون قصد نانوشته، مبهم و اغلب در میانه نشست متغیر است. و همان چیزی است که کاربر نهایتاً عامل را با آن قضاوت می‌کند.
۲	درستی کارکردی	آیا کد ساخته می‌شود، اجرا می‌شود و آزمون‌ها را می‌گذراند؟ کف است، نه سقف. آسان برای سنجش و آسان برای تقلب: آزمون‌ها را می‌شود حذف یا جعل کرد تا قرمز سبز شود، بی‌آنکه چیزی واقعاً درست شده باشد.
۳	درستی بصری و رفتاری	برای عامل‌هایی که برنامه وب یا رابط کاربری تولید می‌کنند، مصنوع همان خروجی رندر شده است، نه کد. سنج‌های سطح کد کاملاً نکته را از دست می‌دهند: صفحه یا درست به نظر می‌رسد و درست رفتار می‌کند، یا نه.
۴	هزینه و کارایی	مصرف توکن، تأخیر واقعی، تعداد فراخوان ابزار و تعداد تکرار: کاربر چند اصلاح باید صادر می‌کرد تا عامل هم‌گرا شود؟ عاملی که در یک نوبت به تغییر درست می‌رسد، محصولی متفاوت از عاملی است که هشت اصلاح لازم دارد.

ابعاد درونی

#	بُعد	شرح
۵	کیفیت کد و تطبیق با قراردادها	آیا کد با اصطلاحات، الگوها و قراردادهای پروژه می‌خواند؟ تغییری که آزمون‌ها را می‌گذرانند اما سبک کدپایه را نقض می‌کند، شکست کدنویسی شهودی است، حتی وقتی موضعاً درست باشد.
۶	کیفیت مسیر	آیا عامل مسیر معقولی رفت: اول فایل‌های مرتبط را خواند، ویرایش‌ها را منسجم مرتب کرد، در هر گام ابزار یا مهارت درست را برگزید؟ خروجی درستی که با استدلال بد تولید شده، موفقیتی شکننده است.
۷	رفتار خودترمیمی	وقتی ساخت شکست می‌خورد، آزمون می‌شکند، یا کاربر می‌گوید «نه»، این‌طور نه»، آیا عامل بهبود می‌یابد یا شکست را انباشته می‌کند؟ کیفیت بازیابی در طول یک نشست چندنوبتی مرکب می‌شود.

محور عرضی: ایمنی و هوش مصنوعی مسئولانه

این محور با چند بُعد تقاطع دارد – آسیب‌پذیری کد، رفتار امتناع، ایمنی محتوا، افشای مالکیت فکری – و باید در کنار هر بُعد سنجیده شود، نه به‌عنوان دروازه‌ای جدا.

و این ابعاد مستقل نیستند: کیفیت مسیر قوی‌تر (بُعد ۶) معمولاً به درستی کارکردی قوی‌تر (بُعد ۲) می‌انجامد، که خودش پیش‌شرط رضایت قصد (بُعد ۱) است.

۱۱. چگونه بسنجیم: هفت روش

هفت بُعد را نمی‌توان با یک روش سنجید. خط لوله‌های تولیدی چند روش را با هم ترکیب می‌کنند:

روش	شرح	بُعد‌های پیشنهادی
آزمون کارکردی خودکار	اجرای ساخت، مجموعه‌آزمون و واری‌گرها روی خروجی عامل، وصل‌شده به خط لولهٔ پروژه. ارزان‌ترین سیگنال موجود.	۲، و بخش‌های قاعده‌پذیر ۵
ارزیابی امنیت و ایمنی	ترکیب تحلیل ایستای امنیتی روی کد تولیدشده با کاوش خصمانه برای سنجش رفتار امتناع. پویشگرهای ایستا آسیب‌پذیری‌ها را پیدا می‌کنند، ابزارهای ویژه نشت اعتبارنامه را می‌گیرند، و مجموعه‌های اسکرپیت‌شدهٔ تیم قرمز می‌آزمایند که آیا عامل درخواست‌های آشکارا مضر را رد می‌کند.	عرضی – کنار همهٔ ابعاد
داور مدل‌زبانی و عامل‌داور	نمره‌دهی خروجی‌ها در برابر سنجه‌ها، برای ابعادی که قاعده‌ها پاسخ درست را نمی‌گیرند. در عمل یعنی مدلی که خروجی را در برابر درخواست اولیهٔ کاربر نمره می‌دهد، یا عامل‌داوری که رد اجرا را برای انسجام برنامه بازرسی می‌کند.	۱، ۵، ۶
آزمون مرورگرمحور	اجرای گردش‌کارهای چندگامی روی برنامهٔ مستقرشده و مشاهدهٔ نتیجه: اسکرپیت‌های تعامل با رابط رندرشده و مقایسهٔ تصویر با مرجع.	۳ (عامل‌های تولیدکنندهٔ رابط)
بازرسی مسیر	تحلیل استدلال عامل، فراخوان ابزارها، فراخوانی مهارت‌ها و بازیابی‌ها. بستر کار، ردهای تله‌متری با دادهٔ فراخوان ابزار در سطح دامنه است که با ابزارهای بازپخش رد نمایش داده می‌شوند و هر فراخوان مدل را به کنش‌های پس از آن گره می‌زنند.	۶، ۷
بازبینی انسانی	نمونه‌گیری از نشست‌ها برای بازبینی مستقیم بازبینان واجد شرایط. مقیاس‌پذیر نیست و عمدتاً برای کالیبره‌کردن بقیهٔ روش‌ها به کار می‌رود. در عمل یعنی حاشیه‌نویسی ساختارمند مهندسان ارشد	۱ (تنها مرجع حقیقت)، ۵، قضاوت‌های ظریف ایمنی

روش	شرح	بُعد‌های پیشنهادی
	روی صف‌های بازبینی پرشده از نمونه‌گیری برخط.	
ارزیابی برخط	نمونه‌گیری از ترافیک زندهٔ تولید و نمره‌دهی با همان سنج‌های برون‌خط. همهٔ ابعاد را با نرخ نمونه‌گیری پوشش می‌دهد.	همه

ترفند نمونه‌گیری در ارزیابی برخط

نمونهٔ یکنواخت یک‌درصدی، دنبالهٔ بلند را از دست می‌دهد. سوگیری را به سمت نشست‌های پرهزینه، نشست‌های با اصلاح‌های متعدد، و نشست‌هایی که کاربر رهايشان کرده ببرید.

محک‌های استاندارد و دام بیش‌برازش

در حالی که چارچوب‌های ارزیابی سفارشی با ماهیت باز کدنویسی شهودی سر و کار دارند، آزمون استاندارد قابلیت‌های شناختی مشخص را از نویز محیط‌های سفارشی سازمانی جدا می‌کند و مبنای تجربی لازم برای اعتماد به سامانه‌ای غیرقطعی را فراهم می‌کند. محک‌ها عامل شما را با میدان رقابت روی مجموعه‌کارهای مشترک مقایسه می‌کنند.

موازنه: بیش‌برازش محک در برابر قصد واقعی

اتکای بیش‌ازحد به محک‌های استاندارد موازنهٔ شدیدی وارد می‌کند. عامل‌ها را می‌توان چنان بهینه کرد که روی مجموعه‌داده‌های ایستا امتیاز درجه‌یک بگیرند اما در مواجهه با واقعیت آشفته و متناقض قصد انسانی در تولید، فاجعه‌بار شکست بخورند. امتیاز بالا در محک مخزن‌های واقعی ثابت می‌کند عامل می‌تواند در یک مخزن بسیار ساختارمند حرکت کند، اما هیچ تضمینی نمی‌دهد که قضاوت زیبایی‌شناختی لازم برای ساخت یک برنامهٔ مصرفی را دارد. محک‌ها را صرفاً برای **کالیبراسیون شناختی** به کار ببرید، نه جایگزین ارزیابی قصد سفارشی.

رصدپذیری، پیش‌شرط ارزیابی است

- **ردیابی اندیشه:** دامنهٔ نشست کل مدت کار را می‌گیرد، دامنهٔ اندیشه چرخهٔ استدلال و درخواست‌دهی پیش از کنش را ثبت می‌کند، و دامنهٔ ابزار آرگومان‌ها و تأخیر تعامل‌های محیطی را.

- **ردیابی هزینه:** با تجمیع ویژگی‌های دامنه‌ها، تیم‌ها می‌توانند مصرف توکن، تأخیر استنتاج و هزینه حلقه‌های خودترمیمی را دقیق اندازه بگیرند.
- **نمونه‌گیری دم‌محور پویا:** گرفتن صد درصد ردها در تولید، بودجه ذخیره‌سازی را می‌بلعد. نمونه‌گیری دم‌محور اجازه می‌دهد جمع‌کننده پس از پایان، کل رد را ارزیابی کند و موفقیت‌های روزمره را دور بریزد و ردهای دارای خطا یا حلقه خودترمیمی افراطی را نگه دارد.

۱۲. نکته‌های کاربردی

پیشوند نشست را سنجۀ قصد کنید

کدنویسی شهودی مشخصاتی برای آزمودن ندارد؛ قصد کاربر نانوشته است و در طول نوبت‌ها تکامل می‌یابد. نزدیک‌ترین چیز به مشخصات، یک یا دو پیام نخست کاربر است. آن‌ها را به‌عنوان سنجۀ در نظر بگیرید: معیارهای ارزیابی را خودکار از پیشوند نشست استخراج کنید و سپس هر نوبت بعدی را در برابر آن‌ها نمره بدهید. این تنها راه عملی برای سنجش رضایت قصد در مقیاس است.

```
# Derive criteria from the user's opening turns
opening = " ".join(session.user_messages[:2])
criteria = client.models.generate_content(
    model="...",
    contents=f"Produce 3-5 acceptance criteria for: {opening}. Return JSON.",
).parsed["criteria"]

# Score every agent turn against the derived criteria
score = client.models.generate_content(
    model="...",
    contents=f"Does this output satisfy {criteria}? Score 1-5 with rationale."
    f"Output: {agent_response}",
).parsed
```

مصنوع رندر شده را داوری کنید، نه کد را

در کدنویسی شهودی کاربر خروجی را قضاوت می‌کند، نه تغییرات کد را. مدلی چندوجهی که به صفحه رندر شده نگاه می‌کند مشکلاتی را می‌گیرد که ارزیابی سطح کد کاملاً از دست می‌دهد: چیدمان شکسته در موبایل، کنتراست ناکافی برای دسترس‌پذیری، وضعیت اشتباه دکمه‌ها. این را با ادعاهای آزمون مرورگر محور جفت کنید: داور مسائل بصری و طراحی را می‌گیرد و ادعاها تعامل‌های شکسته را.

هم‌گرایی نشست را بسنجید، نه درستی تک‌نوبتی

نشست کدنویسی شهودی ذاتاً چندنوبتی است. پرسش مربوط این نیست که «آیا نوبت چهارم درست بود؟» بلکه این است که «آیا کاربر به چیزی که می‌خواست هم‌گرا شد؟» نشست‌هایی که در چند نوبت کم هم‌گرا می‌شوند موارد موفقیت‌اند. نشست‌هایی که کاربر در میانه رهایشان کرده، آموزنده‌ترین شکست‌ها هستند – به‌مراتب آموزنده‌تر از خطاهای تک‌نوبتی.

```
def session_outcome(trace):  
    return {  
        "converged": trace.last_turn.user_signal == "satisfied",  
        "turns_to_converge": trace.user_correction_count,  
        "abandoned": trace.last_user_action == "close",  
        "cost_to_converge": trace.total_token_cost_usd,  
    }
```

اصلاح‌های کاربر را به‌عنوان دادهٔ برچسب‌خورده استخراج کنید

هر «نه، این‌طور نه» از سوی کاربر یک نمونهٔ شکستِ برچسب‌خورده است و کدنویسی شهودی این‌ها را به‌وفور تولید می‌کند. آن‌ها را جمع کنید، به بردار تبدیل کنید و خوشه‌بندی کنید: شکاف‌های نظام‌مند عامل بسیار سریع‌تر از ساختن یک محک شکست مصنوعی دیده می‌شوند. خوشه‌های حاصل، فهرست اولویت‌بندی‌شدهٔ حالت‌های شکست برای تکرار بعدی‌اند.

دو حوزهٔ عمدهٔ بیرون از این چارچوب

ارزیابی ذهنی خروجی‌های راستی‌آزمایی‌ناپذیر، جایی که کیفیت را ترجیح کاربر یا سازمان تعریف می‌کند نه حقیقت زمینی؛ و حلقهٔ بازخورد از اصلاح‌های کاربر به مدل، مهار یا مجموعهٔ ارزیابی برای هدایت بهبود. هر دو موضوعی مستقل‌اند.

۱۳. خودآزمایی

1. تفاوت امنیت و ارزیابی را در یک جمله بگویید و برای هرکدام یک شکست واقعی از تجربهٔ خودتان مثال بزنید.
2. در سازمان شما، هر یک از هفت ستون چه کسی را مالک دارد؟ کدام ستون بی‌مالک است؟
3. آخرین باری که عامل بسته‌ای نصب کرد، چطور مطمئن شدید آن بسته واقعاً وجود دارد و همانی است که فکر می‌کنید؟

4. کد تولیدشدهٔ اخیر شما کدام یک از دو الگوی شکست را دارد؟ چطور می‌سنجید؟
5. عامل شما امروز با چه هویتی کار می‌کند: تفویضی یا عامل‌محور؟ اگر تفویضی است، چه چیزی مانع معاون گیج می‌شود؟
6. آخرین کنش پرمخاطره‌ای که تأیید کردید چه بود؟ آیا واقعاً می‌دانستید چه چیزی را تأیید می‌کنید؟
7. اگر امروز عامل شما منحرف شود، چه چیزی آن را می‌گیرد و در چند ثانیه؟
8. سه دلیل متمایز بودن ارزیابی کدنویسی شهودی را نام ببرید و بگویید کدام برای پروژهٔ شما بحرانی‌تر است.
9. برای هر یک از هفت بُعد، بگویید امروز کدام یک را می‌سنجید و کدام را نه.
10. از ده نشست اخیرتان، چند تا هم‌گرا شدند و چند تا رها شدند؟ رهاشده‌ها چه الگوی مشترکی دارند؟

منبع: این جزوه خلاصه و بازنویسی آموزشی مقالهٔ Vibe Coding Agent Security and Evaluation نوشتهٔ سوکراتیس کارتاکیس، آرون آیدلمن، وفاء بکالی و ملتم سوباشی‌اوغلو است (مجموعهٔ مقالات عامل‌ها، می ۲۰۲۶). ترجمهٔ کامل فارسی در همین پوشه در دسترس است.
